E2A

Enterprise-to-Agentic Architecture Framework

A structured methodology for translating enterprise architecture expertise into production-grade agentic AI systems

---

Subham Gupta

Staff Architect & AI Architect | SAP Labs India

13+ years enterprise systems | $350M+ financial scale

github.com/subhamviky | subhamviky@gmail.com | April 2026

Executive Summary

Enterprise architects possess deep, production-proven expertise in layered system design, distributed transaction management, domain modelling, and governance frameworks. Yet when tasked with building agentic AI systems, many treat this expertise as irrelevant — starting from scratch with tutorials and prototype code, producing systems that are neither architecturally sound nor production-ready.

The Enterprise-to-Agentic (E2A) Architecture Framework challenges this assumption directly. E2A demonstrates that the structural layers underpinning enterprise frameworks — SAP ABAP OOP, SAP RAP, Oracle SOA, mainframe service architectures — are expressions of Clean Architecture: the same foundational pattern that governs every well-structured Python microservice, Spring Boot application, and Lang Graph agent system.

The core thesis: the runtime changes; the architecture does not. An enterprise architect who understands Behaviour Definitions, Determinations, and CDS entities already understands BaseAgent abstract classes, CriticAgent quality gates, and AgentState TypedDicts — they simply have not seen the mapping made explicit.

What E2A Provides

1. A formal mapping between enterprise architectural concepts and their agentic AI equivalents
2. A production-grade project blueprint covering structure, IaC, observability, CI/CD, NFRs, and business logic
3. An NFR-First development methodology preventing the most common production anti-patterns in agentic AI
4. A calibration principle for when to apply enterprise-grade controls and when they would be over-engineering
5. A portable mental model applicable across SAP, Oracle, Mainframe, and other enterprise backgrounds

1. The Problem E2A Solves

1.1 The False Discontinuity

When enterprise architects encounter agentic AI systems, they typically observe: Python scripts calling LLM APIs, JSON dictionaries passed between functions, and ad-hoc orchestration glue. The conclusion drawn is that AI systems are fundamentally different from enterprise systems — unstructured, experimental, incompatible with enterprise design discipline.

This conclusion is false. What they are observing is not the nature of agentic AI systems — it is the consequence of building them without architectural discipline. The same observation could have been made about early Java enterprise applications before patterns like MVC, Service Layer, and Repository were formalised.

1.2 The Two Failure Modes

Failure Mode	Description and Consequence
Logic-First Development	Agents and RAG pipelines built before infrastructure, observability, or governance are in place. Observability is bolted on after the fact — or never. Groundedness drift and token cost explosion are discovered in production. Governance gaps cause compliance failures.
Pattern Paralysis	Enterprise architects recognise that established patterns should apply but cannot identify the mapping. They either produce over-engineered systems applying unnecessary enterprise controls, or under-engineered systems abandoning architectural discipline entirely.

1.3 What E2A Fixes

- Makes the structural mapping explicit — enterprise architects can navigate agentic AI design using their existing mental models
- Provides a development sequence (NFR-First) that prevents logic-first anti-patterns
- Defines when enterprise-grade controls are justified and when they constitute over-engineering
- Produces systems that are production-ready from first deployment, not after retrofitting

2. The Core Insight — Clean Architecture Across Runtimes

2.1 Clean Architecture Is the Constant

Robert Martin's Clean Architecture defines a layered system in which the data model, interface contracts, business logic, operations, and exposure layer are explicitly separated. This is not a pattern invented by software architecture theorists — it is the pattern that emerges when any sufficiently complex system is designed by engineers who have experienced the pain of coupling.

SAP ABAP OOP, SAP RAP, Spring Boot microservices, FastAPI applications, and LangGraph agent systems all express Clean Architecture. The layer names differ; the structural relationships do not.

2.2 The E2A Mapping Table — Ten Architectural Layers

Architectural Layer	SAP RAP / ABAP	Spring Boot	FastAPI / Python	LangGraph Agent
Data model	CDS View Entity	@Entity JPA class	Pydantic BaseModel	AgentState TypedDict
Interface contract	Behavior Definition (BDEF)	Java interface	Protocol / ABC	BaseAgent abstract class
Business logic	Behavior Implementation (BIMP)	@Service class	Concrete service class	Agent class + LLM calls
Callable operation	RAP Action / BOPF Action	@Transactional method	Route handler function	AgentTool / MCP Tool
Auto quality gate	Determination	Spring @Aspect / AOP	Middleware / decorator	CriticAgent
Exposure layer	OData Service Binding	@RestController	FastAPI APIRouter	FastAPI /workflow endpoint
Async events	BOPF Event Framework	Spring Kafka @KafkaListener	SQS worker / consumer	LangGraph conditional edge
Observability	CDS Analytical Views + ATC	Micrometer + Actuator	structlog + OTel + CloudWatch	observability/ (tracer, metrics, logger)
Governance	Authorization Objects + Validations	Spring Security + @Validated	policies/ YAML + OPA	CriticAgent SLO gate + policy engine
Deployment	SAP CTS+ transport	Docker + Helm + kubectl	ECS Fargate + GitHub Actions	GitHub Actions -> ECR -> ECS deploy

2.3 Why Agentic AI Is Called Out Specifically

Spring Boot microservices and FastAPI applications are routinely built with explicit layering — the frameworks enforce it. Agentic AI systems are not. The majority of production agentic AI systems in operation today consist of a single Python file with functions calling LLM APIs, no interface contracts, no quality gates, and no explicit data model. The consequences are: unpredictable behaviour at scale, untestable components, and no governed path to production.

E2A names the mapping explicitly for agentic AI because that is where the gap is widest and the consequences of ignoring it are most severe.

3. NFR-First Development Methodology

3.1 The Sequence

The E2A methodology prescribes a specific development sequence. Business logic — agents, RAG pipelines, tool microservices — is built last. Everything that the business logic depends on to behave correctly in production is built first.

Phase	What Gets Built	Enterprise Equivalent	Why First
1. Scaffold	Folder structure, dependency management (Poetry), pre-commit hooks	ABAP package + transport setup	Forces architectural decisions before any code exists
2. IaC	Terraform: ECS, ALB, API Gateway, SQS/DLQ, OpenSearch, Secrets Manager	SAP system landscape (DEV -> QA -> PROD)	Infrastructure reproducible and auditable from day one
3. Observability	Structured logging, OTel tracing, CloudWatch metrics, SLO checker, error budget	CDS analytical views + ATC runtime monitoring	Every feature deployed into a monitored environment
4. CI/CD	GitHub Actions: OIDC auth, RAGAS eval gate, Trivy scan, auto-rollback on failure	SAP CTS+ transport with ATC approval gates	Quality gates enforced before any logic ships
5. NFRs	Circuit breakers, retry/backoff, FinOps token routing, policy-as-code, LLM cache	ABAP Unit test harness + authorisation framework	Resilience and governance baked in, not bolted on
6. Business Logic	Agents, RAG pipeline, tool microservices	BIMP classes, determinations, actions	Built on a production-ready foundation from day one

3.2 The Terraform Lifecycle as a Test Pattern

Infrastructure-as-Code (Terraform) follows the same four-phase lifecycle as a unit test framework. This is not metaphor — both patterns exist to produce reproducible, auditable, controlled environments from parameterised inputs.

Phase	Terraform	Unit Test Framework (ABAP AUnit)
Setup	terraform init — downloads providers, initialises state backend	SETUP method — initialises test harness, framework readiness
Controlled Inputs	variables.tf — parameterised config (region, CPU, model IDs, limits)	Test doubles / mock objects — controlled inputs isolating business logic
Execution	terraform apply — provisions all cloud resources	TEST method — runs business logic under test
Assertion	outputs.tf + smoke tests — exposes endpoints, verifies health responses	ASSERT statements — verifies expected results against actual output

Important Qualification

Terraform provisions real cloud infrastructure, not mocks. The analogy holds at the lifecycle pattern level — both frameworks enforce reproducible, parameterised, auditable environments through the same four phases. The comparison breaks down if interpreted as 'Terraform is a mock.' The correct framing: Terraform does for infrastructure what unit tests do for business logic. Both produce predictable, reproducible results from controlled inputs.

4. Financial Integrity Patterns — Correct by Design

4.1 Idempotency and Reconciliation as Business Features

The most critical insight from enterprise financial systems is that idempotency and reconciliation are not technical safeguards — they are business contracts. The appropriate architectural weight of these controls is determined by the complexity of the domain, not by engineering convention.

4.2 The Complexity Calibration Principle

Three factors determine whether financial integrity controls belong at the business logic layer or can be handled at the infrastructure layer:

- Input mutability: can amounts, rates, or quantities change legitimately after the process starts?
- Output ambiguity: can two records with the same type and value carry different business meaning?
- Regulatory and audit exposure: what is the financial, legal, or reputational cost of a wrong posting?

Payment Scenario	Complexity Score	Appropriate Layer	Enterprise Pattern Applied
SAP TM Charge Management — rate tables, calculation levels, revised carrier invoices, dispute management	VERY HIGH	Business logic	REF_ELEM_KEY deterministic mapping, Completely Invoiced gate, first-class Dispute workflow
Distributed financial settlement — multi-service saga, cross-ledger posting, regulatory reporting	HIGH	Business logic	Saga + compensation, @Idempotent AOP, double-entry DB invariant, RAG audit
E-commerce checkout — fixed price, single charge type	MEDIUM	API layer	Idempotency key header, payment gateway dedup, standard retry
SaaS subscription billing — fixed monthly, same account	LOW	Infrastructure layer	SQS message deduplication ID, exactly-once delivery setting

4.3 The SAP TM Origin — REF_ELEM_KEY and the Completely Invoiced Gate

In SAP Transportation Management Charge Management, financial integrity is enforced structurally, not defensively:

- The Line-Element Key (REF_ELEM_KEY) creates a deterministic one-to-one mapping between every source charge and its settlement posting. When a carrier revises an invoice amount, the system routes it as a valid business update — not a duplicate — because the key encodes business identity, not a technical hash.
- The Completely Invoiced status gate means the Finance Ledger cannot receive a posting until the business has explicitly confirmed the invoice is final. The ledger is immutable by architectural contract — not by a try/catch block.
- Dispute Management is a first-class business workflow that mediates charge deltas and unblocks final postings. Discrepancies are expected; the architecture provides a governed path to resolve them.

SAP TM Mechanism	E2A Cloud Equivalent	Governing Principle
REF_ELEM_KEY — business identity key	Client-supplied X-Idempotency-Key header + Redis SETNX @Idempotent AOP	Business identity, not infrastructure hash

SAP TM Mechanism	E2A Cloud Equivalent	Governing Principle
Completely Invoiced gate — business status blocks ledger write	SettlementState.COMPLETED as sole pre-condition for ledger finalisation + state machine guard	Ledger immutable by architectural contract
Dispute Management — first-class reconciliation workflow	RAG-powered CriticAgent / Audit-AI service reasoning against policy documents	Discrepancies governed, not silenced
FI Ledger — write-once source of truth	UNIQUE INDEX on (settlement_id, direction, entry_type) + reversal-only corrections	Immutability enforced at DB level
Revised amount = valid update	Idempotency cache returns stored response for known key — retry with same key gets original result	Business update path, not conflict error

5. Domain-by-Domain Mapping Reference

5.1 Agent Architecture

Python Component	ABAP OOP Equivalent	RAP Equivalent	Role
AgentState TypedDict	Global DDIC Structure	CDS View Entity	Central data model passed across all components
IF_AGENT interface	ABAP Interface (IF_*)	Behavior Definition (BDEF)	Contract — declares operations all agents must implement
BaseAgent abstract class	Abstract Superclass	Abstract BIMP	Shared LLM setup, token tracking — not directly instantiated
RouterAgent	Inherited class — classifyIntent	Concrete BIMP — intent determination	Routes query to correct downstream agent
KnowledgeAgent	Inherited class — retrieveAndRespond	Concrete BIMP — RAG-based action	Grounded Q&A with mandatory citations
OrderOpsAgent	Inherited class — processOrder	Concrete BIMP — transactional action	Order creation, stock checks, idempotent writes
FinanceAgent	Inherited class — evaluateRisk	Concrete BIMP — approval/validation	Risk scoring, approval gates, escalation routing
CriticAgent	Validation class	Post-processing Determination	Auto-triggered groundedness scoring; SLO >= 0.85 enforced
LangGraph StateGraph	Workflow dispatcher	Determinations + Actions sequence	Conditional routing between agents on AgentState
FastAPI APIRouter	Service binding	Service Definition + Binding	Exposes agent workflow as OData-equivalent HTTP endpoint

5.2 RAG Pipeline

Python Component	ABAP OOP	RAP Managed	Purpose
rag/index_config.py	Abstract Persistence Class	CDS Entity + DDL Source	Defines OpenSearch schema (KNN vector + BM25 text)
rag/embeddings.py	Utility class — feature extraction	Determination for derived fields	Converts text to semantic vectors (SentenceTransformer / Bedrock Titan)
rag/chunker.py	Data transformation class	Behavior Determination for child entities	Splits documents into overlapping chunks with metadata
rag/indexer.py	Controller class — orchestrates ingestion	Action class — bulk ingestion operation	Read docs -> chunk -> embed -> index to OpenSearch
rag/retriever.py	Search class — multi-strategy	Determination for ranked results	Hybrid: BM25 + KNN -> Reciprocal Rank Fusion
rag/reranker.py	Validation class — result quality	Determination for precision refinement	Cross-encoder reranker improves top-k precision
rag/evaluator.py	Test class	ATC check / Unit test	RAGAS: faithfulness, relevancy, precision, recall

5.3 Tool Call Architecture

Python Component	ABAP OOP	RAP Managed	Purpose
tools/base_tool.py	Abstract Class	Abstract BIMP	Idempotency (DynamoDB lock), structured logging, metrics
tools/create_order.py	Inherited class — write operation	Action: createOrder()	Order creation with approval threshold, DynamoDB persistence
tools/check_stock.py	Inherited class — read operation	Determination: checkStock()	Inventory read — deterministic, no side effects
tools/risk_score.py	Utility class — evaluation	Determination: evaluateRisk()	Risk scoring — returns recommendation, no write
tools/open_case.py	Inherited class — write operation	Action: openCase()	Case creation with SLA assignment and idempotency
FastAPI tool router	Facade class	Service Definition + Binding	Aggregates all tool endpoints under one API surface

5.4 Observability — AI-Specific KPIs

Standard application observability (latency, error rate, resource utilisation) is necessary but insufficient for production agentic AI systems. E2A mandates four additional KPI dimensions:

KPI	Implementation	SLO Target	Alert Condition
Groundedness score	RAGAS faithfulness per workflow	>= 0.85	< 0.85 blocks deployment; < 0.70 triggers SLO freeze
Token cost per workflow	CloudWatch PutMetricData — tokens x model price	< $0.03 / workflow	> $0.05 triggers FinOps alert; model routing switches to cheaper model
Tool reliability rate	Success/failure counter per tool	>= 99%	< 98% activates circuit breaker; DLQ depth > 10 triggers alert
End-to-end latency	OTel span: API Gateway -> response	p95 < 2.5s	> 3.0s triggers SLO freeze; deployment paused

Observability Component	AWS / Python Implementation	Enterprise Equivalent
Structured Logging	structlog JSON -> CloudWatch Logs Insights	CDS Audit Tables + RAP Logging Framework
Distributed Tracing	OTel SDK + AWS X-Ray spanning API -> agents -> tools	RAP Determination trace + SAP Cloud ALM
Custom Metrics	CloudWatch PutMetricData: latency, groundedness, tokens, cost	RAP Monitoring + ABAP Runtime Statistics + CDS KPI Views
SLO Enforcement	slo_checker.py runs after each workflow; flags violations	RAP Determinations + Validation Classes post-processing
Error Budget	error_budget.py: cumulative SLO consumption; sprint trigger at 50%	ATC dashboard + remediation cycle governance
RAGAS CI/CD Gate	rag_eval.yml nightly: blocks deploy if faithfulness < 0.85	ATC transport release check — fails transport on quality breach

5.5 NFR Triad — Resilience, FinOps, Governance

E2A defines three NFR pillars that must be provisioned before business logic:

Resilience

- Circuit breakers per downstream service — prevents cascading failures (pybreaker + CloudWatch metrics)
- Retry with exponential backoff and jitter — avoids thundering herd on transient failures (tenacity decorator)
- Fallback modes — safe defaults when a tool or LLM endpoint is unavailable
- Async SQS worker — offloads long-running workflows from synchronous request path

FinOps

- Token budget per workflow — enforced as a typed dataclass with per-model pricing
- Dynamic model routing — complex tasks use Claude Sonnet; simple classification uses Claude Haiku
- Semantic LLM cache — DynamoDB TTL cache keyed by SHA256 of query + model ID reduces token spend
- Cost tracking — CloudWatch metric for token usage and dollar cost per workflow intent

Governance

- Policy-as-Code (YAML/OPA) — tool allow/deny lists, approval gates, redaction rules defined declaratively
- Approval gates — orders above threshold require FinanceAgent escalation before tool invocation
- Redaction pipeline — PII masking applied before any data reaches LLM context
- SLO freeze — CI/CD deployment blocked automatically when error budget consumption exceeds 50%

6. CI/CD Pipeline — Quality Gates at Every Stage

Pipeline Stage	Implementation	Enterprise Equivalent	Gate Condition
Lint + pre-commit	Black, Ruff, YAML validation, Terraform fmt	ATC syntax check + ABAP formatter	Blocks commit on any violation
Unit tests	pytest + coverage >= 70%	ABAP Unit — class-level smoke tests	Blocks PR merge on failure or coverage drop
Policy compliance	Governance YAML validation + Trivy IaC scan	ATC transport check + authorisation review	Blocks PR merge on policy violation
RAG evaluation	RAGAS: faithfulness, relevancy, precision, recall	Custom AUnit for semantic correctness	Blocks deploy if any score < 0.85
Build + scan	Docker build -> Trivy vulnerability scan -> ECR push	Transport packaging + security review	Blocks deploy on HIGH/CRITICAL CVE
Deploy + rollback	ECS service update -> smoke test -> auto-rollback on failure	Transport release with rollback procedure	Auto-rollback if health check fails within 5 minutes

OIDC Authentication — No Long-Lived Credentials

GitHub Actions assumes an AWS IAM role via OIDC federation — no static access keys stored in secrets. Equivalent to SAP OAuth 2.0 service-to-service authentication — short-lived tokens, scoped permissions. This is the correct security posture for CI/CD pipelines in regulated environments.

7. Applicability — Beyond SAP

7.1 The Framework is Background-Agnostic

E2A was developed from SAP RAP and ABAP OOP experience because that is the background of its author. However, the structural mapping is not SAP-specific — it applies to any enterprise background where layered architecture was used.

Enterprise Background	Equivalent Concepts	E2A Mapping Entry Point
SAP ABAP / RAP	BDEF, BIMP, CDS Entity, Determination, Action, BOPF	Direct — all six layers map explicitly (see Section 2.2)
Java EE / Spring	JPA Entity, Service interface, @Service, @Aspect, @RestController	Near-direct — Spring Boot column in mapping table
Oracle SOA / ADF	Business Service, Entity Object, View Object, Application Module, Task Flow	Entity Object -> AgentState; Business Service -> BaseAgent; Task Flow -> LangGraph graph
Mainframe / COBOL	COPYBOOK (data layout), PROCEDURE DIVISION, CALL interface, batch JCL	COPYBOOK -> TypedDict; CALL interface -> abstract class; JCL -> Terraform + GitHub Actions
MuleSoft / Integration	Mule Flow, DataWeave transform, Connector, Error Handler, API Gateway	Flow -> LangGraph graph; Connector -> AgentTool; DataWeave -> Pydantic schema

7.2 Cloud Platform Portability

All E2A patterns are vendor-agnostic at the architectural level. The AWS-specific implementations shown in this document have direct equivalents on GCP and Azure:

E2A Component	AWS Implementation	GCP Equivalent	Azure Equivalent
Container hosting	ECS Fargate	Cloud Run	Azure Container Apps
Async messaging	SQS FIFO + DLQ	Cloud Pub/Sub + Dead Letter	Service Bus + DLQ
Vector store	OpenSearch (KNN)	Vertex AI Vector Search	Azure AI Search
LLM inference	Amazon Bedrock	Vertex AI (Gemini)	Azure OpenAI Service
Secret management	Secrets Manager	Secret Manager	Key Vault
Observability	CloudWatch + X-Ray	Cloud Monitoring + Cloud Trace	Azure Monitor + App Insights
IaC	Terraform (AWS provider)	Terraform (GCP provider)	Terraform (Azure provider)

8. Quick Reference Card

8.1 The Six Core Mappings

Enterprise Concept	Agentic AI Equivalent	Key Invariant
CDS View Entity / JPA @Entity	AgentState TypedDict	Typed central data model — one instance per workflow
Behavior Definition / Java interface	BaseAgent abstract class	Interface contract — all agents must implement __call__
Behavior Implementation / @Service	Specialized Agent class	One concrete class per domain (Router, Knowledge, etc.)
RAP Action / @Transactional method	AgentTool / MCP Tool	Idempotency key + DynamoDB conditional write for write tools
Determination / Spring @Aspect	CriticAgent	Auto-triggered — caller does not invoke; graph routes to it
OData Service Binding / @RestController	FastAPI APIRouter	Stateless exposure — no business logic in the router layer

8.2 The NFR-First Sequence

1. 1. Scaffold — folder structure, Poetry, pre-commit hooks
2. 2. IaC — Terraform: ECS, ALB, SQS, OpenSearch, Secrets
3. 3. Observability — structlog, OTel/X-Ray, CloudWatch, SLO checker
4. 4. CI/CD — GitHub Actions: OIDC, RAGAS gate, Trivy, auto-rollback
5. 5. NFRs — circuit breakers, FinOps routing, policy-as-code, LLM cache
6. 6. Business Logic — agents, RAG pipeline, tool microservices

8.3 The Calibration Principle

Factor	Score HIGH → Business layer	Score LOW → API/Infra layer	Decision
Input mutability	Amounts / quantities change mid-flight	Fixed at creation	HIGH: business identity key required
Output ambiguity	Same value, different business meaning by context	Value uniquely identifies record	HIGH: value-hash dedup is insufficient
Audit exposure	Regulatory reporting, vendor payments, legal	Internal metrics, logging	HIGH: immutable ledger + compensation required

8.4 The Terraform Lifecycle

Phase	Terraform	Unit Test Framework
Setup	terraform init	SETUP method
Controlled Inputs	variables.tf	Test doubles / mock objects
Execution	terraform apply	TEST methods
Assertion	outputs.tf + smoke tests	ASSERT statements

9. About the Author

Subham Gupta is a Staff Architect and AI Architect at SAP Labs India with 13+ years of production systems delivery. He governs $350M+ in annual financial transaction volumes and leads mission-critical distributed settlement systems in SAP Transportation Management.

The E2A Framework was developed from hands-on experience spanning two domains: enterprise financial systems at production scale, and cloud-native AI portfolio projects built from first principles on AWS. The framework is validated through three active production-grade projects:

- Cloud-Native Financial Settlement Platform — Java 21, Spring Boot 3.x, Kafka, Redis, Spring AI + RAG, AWS EKS
- Order-to-Cash Agentic AI Platform — Python, LangGraph, Amazon Bedrock, OpenSearch hybrid RAG, Terraform IaC
- Cloud-Native Payment Reconciliation Engine — Python, FastAPI, AWS Lambda, SQS, DynamoDB, Amazon Bedrock

Contact	Link
GitHub (portfolio projects)	github.com/subhamviky
LinkedIn	linkedin.com/in/subham-gupta-0a05a058
Email	subhamviky@gmail.com